

Lesson 9 Notes: Upper Triangular Matrix Mappings

1. Main idea of Lesson 9

Lesson 9 teaches **upper triangular matrix mappings**.

Before this lesson, you studied **lower triangular matrices**.

A lower triangular matrix stores values on and below the main diagonal:

```
i >= j
```

Lesson 9 changes the shape.

An **upper triangular matrix** stores values on and above the main diagonal:

```
i <= j
```

So Lesson 9 is about:

- what an upper triangular matrix is
- which positions are stored
- which positions are structural zeros
- how many values are stored
- row-major mapping
- column-major mapping
- dry runs for index formulas
- helper functions for getting values
- relationship with lower triangular formulas
- time and space complexity

The most important idea is this:

```
Upper triangular matrix = useful values are in the top-right triangle.
```

2. Current progress review

So far, the course has moved like this:

Lessons	Matrix type / idea	Main condition
Lessons 1–4	Diagonal matrix	<code>i == j</code>
Lesson 5	Lower triangular row-major mapping	<code>i >= j</code>
Lesson 6	Lower triangular column-major mapping	<code>i >= j</code>
Lesson 7	Lower triangular matrix in C	<code>i >= j</code>
Lesson 8	Lower triangular matrix in C++	<code>i >= j</code>
Lesson 9	Upper triangular matrix mappings	<code>i <= j</code>

So Lesson 9 is not starting from zero.

It uses the same general storage idea:

Store only useful values in a 1D array.
Return or print zero for structural zero positions.

The main change is the matrix shape.

3. Quick reminder: what is the main diagonal?

The **main diagonal** goes from the top-left corner to the bottom-right corner.

Example:

```
1  0  0  0
0  5  0  0
0  0  9  0
0  0  0 13
```

The main diagonal positions are:

```
M[1,1]
M[2,2]
M[3,3]
M[4,4]
```

For every main diagonal position:

```
i == j
```

In an upper triangular matrix, the main diagonal is included in the stored region.

4. What is an upper triangular matrix?

An **upper triangular matrix** is a square matrix where the useful values are on or above the main diagonal.

Example:

```
1  2  3  4  5
0  6  7  8  9
0  0 10 11 12
0  0  0 13 14
0  0  0  0 15
```

The useful part is:

```
1  2  3  4  5
  6  7  8  9
    10 11 12
      13 14
        15
```

This useful part forms a triangle in the upper-right part of the matrix.

That is why it is called an **upper triangular matrix**.

5. Stored condition

For an upper triangular matrix, a position is stored when:

```
i <= j
```

Where:

```
i = row number  
j = column number
```

Examples:

Position	Check	Stored?
M[1,1]	1 <= 1	Yes
M[1,2]	1 <= 2	Yes
M[1,5]	1 <= 5	Yes
M[2,2]	2 <= 2	Yes
M[3,5]	3 <= 5	Yes
M[4,5]	4 <= 5	Yes

So the main rule is:

```
if (i <= j)
```

If this condition is true, the position belongs to the upper triangle.

6. Zero condition

For an upper triangular matrix, a position is zero when:

```
i > j
```

This means the position is below the main diagonal.

Examples:

Position	Check	Meaning
M[2,1]	2 > 1	Structural zero
M[3,1]	3 > 1	Structural zero
M[4,2]	4 > 2	Structural zero
M[5,3]	5 > 3	Structural zero

These values are not stored in the array.

The program simply returns or prints `0` when needed.

7. Structural zeros in an upper triangular matrix

A **structural zero** is a zero that exists because of the matrix rule.

In an upper triangular matrix, every value below the main diagonal is a structural zero.

Example:

```
M[4,2] = 0
```

Why?

Because:

```
4 > 2
```

So `(4,2)` is below the main diagonal.

That position is not stored.

The program already knows it must be zero.

8. Stored region vs zero region

For this matrix:

```
1  2  3  4
0  5  6  7
0  0  8  9
0  0  0 10
```

Stored positions are:

```
M[1,1] M[1,2] M[1,3] M[1,4]
      M[2,2] M[2,3] M[2,4]
```

```

M[3,3] M[3,4]
      M[4,4]

```

Zero positions are:

```

M[2,1]
M[3,1] M[3,2]
M[4,1] M[4,2] M[4,3]

```

Stored condition:

```
i <= j
```

Zero condition:

```
i > j
```

9. Comparison with lower triangular matrix

Lesson 9 is easier if you compare it with the lower triangular matrix.

Matrix type	Stored condition	Zero condition	Useful shape
Lower triangular	<code>i >= j</code>	<code>i < j</code>	bottom-left triangle
Upper triangular	<code>i <= j</code>	<code>i > j</code>	top-right triangle

A lower triangular matrix looks like this:

```

1  0  0  0
2  3  0  0
4  5  6  0
7  8  9 10

```

An upper triangular matrix looks like this:

```

1  2  3  4
0  5  6  7

```

```
0 0 8 9
0 0 0 10
```

So upper triangular is like the opposite side of lower triangular.

10. Why upper triangular storage saves memory

A normal $N \times N$ matrix stores:

N^2 values

But an upper triangular matrix stores only:

$N(N + 1) / 2$ values

This is about half of the full matrix.

For example, for $N = 5$:

```
Full matrix values = 5 x 5 = 25
Stored upper triangular values = 5(5 + 1) / 2
                                = 5(6) / 2
                                = 30 / 2
                                = 15
```

So for a 5×5 upper triangular matrix, we store only **15 values** instead of **25 values**.

The remaining values are structural zeros.

11. Number of structural zeros

The number of structural zeros below the main diagonal is:

$N(N - 1) / 2$

For $N = 5$:

$$\begin{aligned}\text{Zeros} &= 5(5 - 1) / 2 \\ &= 5(4) / 2 \\ &= 20 / 2 \\ &= 10\end{aligned}$$

So a **5 x 5** upper triangular matrix has:

15 stored values
10 structural zeros
25 total visible positions

12. Stored values formula

For an **N x N** upper triangular matrix:

$$\text{stored values} = N(N + 1) / 2$$

This formula tells us how large the 1D array should be.

Example for **N = 4**:

$$\begin{aligned}\text{stored values} &= 4(4 + 1) / 2 \\ &= 4(5) / 2 \\ &= 20 / 2 \\ &= 10\end{aligned}$$

So a **4 x 4** upper triangular matrix needs a 1D array of size **10**.

13. Two ways to store upper triangular values

There are two common storage orders:

1. Row-major storage
2. Column-major storage

Both store the same upper triangular matrix.

But the values appear in a different order inside the 1D array.

Part A: Row-Major Upper Triangular Mapping

14. What does row-major mean?

Row-major means we store values row by row.

We start with row 1.

Then row 2.

Then row 3.

Then row 4.

And so on.

For an upper triangular matrix, each row stores values from the diagonal to the right side.

15. Row-major storage example

Matrix:

```
1  2  3  4  5
0  6  7  8  9
0  0 10 11 12
0  0  0 13 14
0  0  0  0 15
```

Row-major storage means:

```
Row 1: 1, 2, 3, 4, 5
Row 2: 6, 7, 8, 9
Row 3: 10, 11, 12
Row 4: 13, 14
Row 5: 15
```

So the 1D array becomes:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

16. Row sizes in upper triangular row-major storage

For an $N \times N$ upper triangular matrix:

```
Row 1 stores N values
Row 2 stores N - 1 values
Row 3 stores N - 2 values
Row 4 stores N - 3 values
...
Row N stores 1 value
```

For $N = 5$:

```
Row 1 stores 5 values
Row 2 stores 4 values
Row 3 stores 3 values
Row 4 stores 2 values
Row 5 stores 1 value
```

Rows become smaller as we move downward.

17. Visual row-major mapping

Using this matrix:

```
1  2  3  4
0  5  6  7
0  0  8  9
0  0  0 10
```

The row-major array is:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Mapping:

Matrix position	Array position
M[1,1]	A[0]
M[1,2]	A[1]
M[1,3]	A[2]
M[1,4]	A[3]
M[2,2]	A[4]
M[2,3]	A[5]
M[2,4]	A[6]
M[3,3]	A[7]
M[3,4]	A[8]
M[4,4]	A[9]

18. Row-major index formula

For upper triangular row-major storage:

$$\text{index} = ((i - 1) * n) - ((i - 2)(i - 1) / 2) + (j - i)$$

Use this formula only when:

$$i \leq j$$

If:

$$i > j$$

then the position is below the diagonal and is not stored.

19. Meaning of the row-major formula

The row-major formula is:

$$\text{index} = ((i - 1) * n) - ((i - 2)(i - 1) / 2) + (j - i)$$

It has two main parts:

$$\text{index} = \text{skipped rows} + \text{movement inside current row}$$

20. Part 1: Skip previous rows

The first part is:

$$((i - 1) * n) - ((i - 2)(i - 1) / 2)$$

This counts how many stored values come before row i .

Example:

Suppose $n = 5$ and we want a value in row 4 .

Before row 4, we have:

Row 1 stores 5 values
Row 2 stores 4 values
Row 3 stores 3 values

So we skip:

$$5 + 4 + 3 = 12 \text{ values}$$

Using the formula:

$$\begin{aligned} & ((i - 1) * n) - ((i - 2)(i - 1) / 2) \\ &= ((4 - 1) * 5) - ((4 - 2)(4 - 1) / 2) \\ &= 3 * 5 - (2 * 3 / 2) \\ &= 15 - 3 \\ &= 12 \end{aligned}$$

So the formula correctly skips 12 values before row 4.

21. Part 2: Move inside the current row

The second part is:

$$j - i$$

This moves inside the current row.

Why?

In an upper triangular matrix, row `i` starts at column `i`.

Example: row 4 starts at:

$$M[4,4]$$

Then the next stored value is:

$$M[4,5]$$

So:

$M[4,4]$ has movement 0
 $M[4,5]$ has movement 1

Using the formula:

$$\text{movement} = j - i$$

For `M[4,5]`:

$$\text{movement} = 5 - 4 = 1$$

22. Row-major formula in plain English

The formula:

```
index = ((i - 1) * n) - ((i - 2)(i - 1) / 2) + (j - i)
```

means:

First, skip all stored values in rows before row i .
Then, move right inside row i until you reach column j .

That is the easiest way to remember it.

23. Row-major dry run 1: Find index of $M[4,5]$

We want:

```
M[4,5]
```

Given:

```
n = 5  
i = 4  
j = 5
```

Step 1: Check condition

```
i <= j  
4 <= 5  
true
```

So $M[4,5]$ is stored.

Step 2: Skip previous rows

Rows before row 4:

```
Row 1 stores 5 values  
Row 2 stores 4 values  
Row 3 stores 3 values
```

So:

```
skippedRows = 5 + 4 + 3  
skippedRows = 12
```

Step 3: Move inside row 4

Row 4 stored positions are:

```
M[4,4], M[4,5]
```

Target is `M[4,5]`, so movement is:

```
j - i = 5 - 4 = 1
```

Step 4: Add both parts

```
index = skippedRows + movement  
index = 12 + 1  
index = 13
```

So:

```
M[4,5] -> A[13]
```

24. Row-major dry run 2: Use the full formula for M[4,5]

Formula:

```
index = ((i - 1) * n) - ((i - 2)(i - 1) / 2) + (j - i)
```

Values:

```
n = 5  
i = 4  
j = 5
```

Substitute:

$$\text{index} = ((4 - 1) * 5) - ((4 - 2)(4 - 1) / 2) + (5 - 4)$$

Simplify:

$$\begin{aligned}\text{index} &= (3 * 5) - ((2)(3) / 2) + 1 \\ \text{index} &= 15 - (6 / 2) + 1 \\ \text{index} &= 15 - 3 + 1 \\ \text{index} &= 13\end{aligned}$$

So:

$$M[4,5] \rightarrow A[13]$$

25. Row-major dry run 3: Find index of M[2,4]

We want:

$$M[2,4]$$

Given:

$$\begin{aligned}n &= 5 \\ i &= 2 \\ j &= 4\end{aligned}$$

Step 1: Check condition

$$\begin{aligned}2 &\leq 4 \\ \text{true}\end{aligned}$$

So the value is stored.

Step 2: Use formula

$$\text{index} = ((2 - 1) * 5) - ((2 - 2)(2 - 1) / 2) + (4 - 2)$$

Simplify:


```
index = (1 * 5) - ((0)(1) / 2) + 2  
index = 5 - 0 + 2  
index = 7
```

So:

```
M[2,4] -> A[7]
```

Check using row-major array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

In the 5 x 5 example:

```
M[2,4] = 8  
A[7] = 8
```

Correct.

26. Row-major dry run 4: Check M[5,2]

We want:

```
M[5,2]
```

Given:

```
i = 5  
j = 2
```

Check condition:

```
i <= j  
5 <= 2  
false
```

So `M[5,2]` is not stored.

Because:

5 > 2

This position is below the main diagonal.

Therefore:

M[5,2] = 0

It is a structural zero.

27. Row-major helper function

A helper function can calculate the row-major array index.

```
int getRowMajorIndex(int n, int i, int j) {  
    if (i <= j) {  
        return ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);  
    }  
    return -1;  
}
```

This function returns:

the array index if the position is stored
-1 if the position is not stored

28. Why return -1 in the helper function?

Valid array indexes start from 0.

Valid indexes are:

0, 1, 2, 3, 4, ...

So -1 is not a valid array index.

That makes it useful as a signal.

It means:

This matrix position is not stored.

Example:

```
getRowMajorIndex(5, 5, 2)
```

Since:

$5 \leq 2$ is false

The function returns:

-1

29. Row-major get function

A `get` function should return the actual matrix value.

```
int getRowMajor(int n, int *A, int i, int j) {  
    if (i <= j) {  
        int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);  
        return A[index];  
    }  
    return 0;  
}
```

Important difference:

Helper index function returns -1 for positions not stored.
Get function returns 0 for positions not stored.

Why?

Because the actual matrix value at a structural zero position is `0`.

30. Row-major set idea

A `set` function stores a value only if the position belongs to the upper triangle.

```
void setRowMajor(int n, int *A, int i, int j, int x) {
    if (i <= j) {
        int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);
        A[index] = x;
    }
}
```

If:

```
i <= j
```

then store the value.

If:

```
i > j
```

then do nothing because the position must remain zero.

31. Row-major display idea

To display the full matrix, we still use nested loops.

The stored array contains only the upper triangle.

But the user should see the full square matrix.

```
void displayRowMajor(int n, int *A) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i <= j) {
                int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);
                cout << A[index] << " ";
            } else {
                cout << "0 ";
            }
        }
    }
}
```

```
    }  
    cout << endl;  
  }  
}
```

For each position:

```
if i <= j, print stored value  
if i > j, print 0
```

Part B: Column-Major Upper Triangular Mapping

32. What does column-major mean?

Column-major means we store values column by column.

We start with column 1.

Then column 2.

Then column 3.

Then column 4.

And so on.

For an upper triangular matrix, each column stores values from the top row down to the diagonal.

33. Column-major storage example

Matrix:

```
1  2  3  4  5  
0  6  7  8  9  
0  0 10 11 12  
0  0  0 13 14  
0  0  0  0 15
```

Column-major storage means:

```
Column 1: 1
Column 2: 2, 6
Column 3: 3, 7, 10
Column 4: 4, 8, 11, 13
Column 5: 5, 9, 12, 14, 15
```

So the 1D array becomes:

```
A = [1, 2, 6, 3, 7, 10, 4, 8, 11, 13, 5, 9, 12, 14, 15]
```

This array represents the same matrix, but the storage order is different from row-major order.

34. Column sizes in upper triangular column-major storage

For an $N \times N$ upper triangular matrix:

```
Column 1 stores 1 value
Column 2 stores 2 values
Column 3 stores 3 values
Column 4 stores 4 values
...
Column N stores N values
```

For $N = 5$:

```
Column 1 stores 1 value
Column 2 stores 2 values
Column 3 stores 3 values
Column 4 stores 4 values
Column 5 stores 5 values
```

Columns become larger as we move to the right.

35. Visual column-major mapping

Using this matrix:

```
1  2  3  4
0  5  6  7
0  0  8  9
0  0  0 10
```

Column-major storage is:

```
Column 1: 1
Column 2: 2, 5
Column 3: 3, 6, 8
Column 4: 4, 7, 9, 10
```

The 1D array is:

```
A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]
```

Mapping:

Matrix position	Array position
M[1,1]	A[0]
M[1,2]	A[1]
M[2,2]	A[2]
M[1,3]	A[3]
M[2,3]	A[4]
M[3,3]	A[5]
M[1,4]	A[6]
M[2,4]	A[7]
M[3,4]	A[8]
M[4,4]	A[9]

36. Column-major index formula

For upper triangular column-major storage:

$$\text{index} = j(j - 1) / 2 + (i - 1)$$

Use this formula only when:

$$i \leq j$$

If:

$$i > j$$

then the position is below the diagonal and is not stored.

37. Meaning of the column-major formula

The column-major formula is:

$$\text{index} = j(j - 1) / 2 + (i - 1)$$

It has two main parts:

$$\text{index} = \text{skipped columns} + \text{movement inside current column}$$

38. Part 1: Skip previous columns

The first part is:

$$j(j - 1) / 2$$

This counts how many stored values come before column `j`.

Example:

Suppose we want a value in column `5`.

Before column 5, we have:

Column 1 stores 1 value
Column 2 stores 2 values
Column 3 stores 3 values
Column 4 stores 4 values

So we skip:

$1 + 2 + 3 + 4 = 10$ values

Using the formula:

$$\begin{aligned} & j(j - 1) / 2 \\ & = 5(5 - 1) / 2 \\ & = 5(4) / 2 \\ & = 20 / 2 \\ & = 10 \end{aligned}$$

So the formula correctly skips 10 values before column 5.

39. Part 2: Move inside the current column

The second part is:

$i - 1$

This moves downward inside the current column.

Why?

In an upper triangular matrix, column j starts from row 1 and goes down to row j .

Example: column 5 has:

$M[1,5]$
 $M[2,5]$
 $M[3,5]$
 $M[4,5]$
 $M[5,5]$

So:

```
M[1,5] has movement 0
M[2,5] has movement 1
M[3,5] has movement 2
M[4,5] has movement 3
M[5,5] has movement 4
```

Using the formula:

```
movement = i - 1
```

For `M[4,5]`:

```
movement = 4 - 1 = 3
```

40. Column-major formula in plain English

The formula:

```
index = j(j - 1) / 2 + (i - 1)
```

means:

```
First, skip all stored values in columns before column j.
Then, move downward inside column j until you reach row i.
```

That is the easiest way to remember it.

41. Column-major dry run 1: Find index of M[4,5]

We want:

```
M[4,5]
```

Given:

```
i = 4  
j = 5
```

Step 1: Check condition

```
i <= j  
4 <= 5  
true
```

So `M[4,5]` is stored.

Step 2: Skip previous columns

Columns before column 5:

```
Column 1 stores 1 value  
Column 2 stores 2 values  
Column 3 stores 3 values  
Column 4 stores 4 values
```

So:

```
skippedColumns = 1 + 2 + 3 + 4  
skippedColumns = 10
```

Step 3: Move inside column 5

Column 5 positions are:

```
M[1,5], M[2,5], M[3,5], M[4,5], M[5,5]
```

Target is `M[4,5]`, so movement is:

```
i - 1 = 4 - 1 = 3
```

Step 4: Add both parts

```
index = skippedColumns + movement  
index = 10 + 3  
index = 13
```

So:

```
M[4,5] -> A[13]
```

42. Column-major dry run 2: Use the full formula for M[4,5]

Formula:

```
index = j(j - 1) / 2 + (i - 1)
```

Values:

```
i = 4  
j = 5
```

Substitute:

```
index = 5(5 - 1) / 2 + (4 - 1)
```

Simplify:

```
index = 5(4) / 2 + 3  
index = 20 / 2 + 3  
index = 10 + 3  
index = 13
```

So:

```
M[4,5] -> A[13]
```

43. Column-major dry run 3: Find index of M[2,4]

We want:

M[2,4]

Given:

i = 2
j = 4

Step 1: Check condition

2 <= 4
true

So the value is stored.

Step 2: Use formula

```
index = j(j - 1) / 2 + (i - 1)
index = 4(4 - 1) / 2 + (2 - 1)
index = 4(3) / 2 + 1
index = 12 / 2 + 1
index = 6 + 1
index = 7
```

So:

M[2,4] -> A[7]

Check using the 4 x 4 column-major array:

A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]

Here:

```
M[2,4] = 7  
A[7] = 7
```

Correct.

44. Column-major dry run 4: Check M[5,2]

We want:

```
M[5,2]
```

Given:

```
i = 5  
j = 2
```

Check condition:

```
i <= j  
5 <= 2  
false
```

So `M[5,2]` is not stored.

Because:

```
5 > 2
```

This position is below the main diagonal.

Therefore:

```
M[5,2] = 0
```

It is a structural zero.

45. Column-major helper function

A helper function can calculate the column-major array index.

```
int getColumnMajorIndex(int i, int j) {  
    if (i <= j) {  
        return (j * (j - 1) / 2) + (i - 1);  
    }  
    return -1;  
}
```

This function returns:

```
the array index if the position is stored  
-1 if the position is not stored
```

46. Why the column-major formula does not need n

The column-major formula is:

```
index = j(j - 1) / 2 + (i - 1)
```

Notice that it does not use `n`.

Why?

Because column-major upper triangular columns grow like this:

```
Column 1 has 1 value  
Column 2 has 2 values  
Column 3 has 3 values  
Column 4 has 4 values  
...
```

To find column `j`, we only need to know how many values came before column `j`.

That count is:

$$1 + 2 + 3 + \dots + (j - 1)$$

And that becomes:

$$j(j - 1) / 2$$

So `n` is not needed for this specific formula.

47. Column-major get function

A `get` function should return the actual matrix value.

```
int getColumnMajor(int *A, int i, int j) {  
    if (i <= j) {  
        int index = (j * (j - 1) / 2) + (i - 1);  
        return A[index];  
    }  
    return 0;  
}
```

If:

$$i \leq j$$

then return the stored value.

If:

$$i > j$$

then return `0`.

48. Column-major set idea

A `set` function stores a value only if the position belongs to the upper triangle.


```
void setColumnMajor(int *A, int i, int j, int x) {
    if (i <= j) {
        int index = (j * (j - 1) / 2) + (i - 1);
        A[index] = x;
    }
}
```

If:

```
i <= j
```

then store the value.

If:

```
i > j
```

then do nothing because the position must remain zero.

49. Column-major display idea

To display the full matrix, use nested loops.

```
void displayColumnMajor(int n, int *A) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i <= j) {
                int index = (j * (j - 1) / 2) + (i - 1);
                cout << A[index] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
}
```

For each position:

```
if i <= j, print stored value
if i > j, print 0
```

Part C: Comparing Row-Major and Column-Major Upper Triangular Storage

50. Same matrix, different array order

Both row-major and column-major storage represent the same matrix.

Matrix:

```
1  2  3  4
0  5  6  7
0  0  8  9
0  0  0 10
```

Row-major array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Column-major array:

```
A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]
```

The visible matrix is the same.

Only the 1D array order changes.

51. Row-major vs column-major comparison table

Concept	Upper row-major	Upper column-major
Matrix type	Upper triangular	Upper triangular
Stored condition	<code>i <= j</code>	<code>i <= j</code>

Concept	Upper row-major	Upper column-major
Zero condition	$i > j$	$i > j$
Storage order	Row by row	Column by column
First stored group	Row 1	Column 1
Group sizes	$N, N-1, N-2, \dots$	$1, 2, 3, \dots$
Formula idea	Skip previous rows	Skip previous columns
Index formula	$((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)$	$j(j - 1)/2 + (i - 1)$
Needs n ?	Yes	No

52. Relationship with lower triangular formulas

This is one of the most important ideas in Lesson 9.

Upper triangular formulas are strongly related to lower triangular formulas.

Lower triangular row-major formula

$$\text{index} = i(i - 1)/2 + (j - 1)$$

Upper triangular column-major formula

$$\text{index} = j(j - 1)/2 + (i - 1)$$

Notice the similarity.

The upper column-major formula looks like the lower row-major formula with i and j swapped.

53. Formula-swapping idea

The project uses this useful idea:

Upper row-major resembles lower column-major with i and j swapped.
 Upper column-major resembles lower row-major with i and j swapped.

This means the formulas are not random.

They are connected.

If you understand lower triangular formulas well, upper triangular formulas become easier.

54. Why formula reuse matters

Formula reuse matters because later programs can be changed more easily.

For example, if you already wrote code for a lower triangular matrix, you can often adapt it for an upper triangular matrix by changing:

1. the condition
2. the index formula

For lower triangular:

```
if (i >= j)
```

For upper triangular:

```
if (i <= j)
```

That is the main logical switch.

Part D: Beginner Coding Ideas

55. Upper triangular row-major get function

```
int getRowMajor(int n, int *A, int i, int j) {  
    if (i <= j) {  
        int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);  
        return A[index];  
    }  
}
```

```
    return 0;
}
```

Explanation:

If $i \leq j$, the value is stored.
Calculate the row-major index.
Return $A[\text{index}]$.
If $i > j$, return 0.

56. Upper triangular column-major get function

```
int getColumnMajor(int *A, int i, int j) {
    if (i <= j) {
        int index = (j * (j - 1) / 2) + (i - 1);
        return A[index];
    }
    return 0;
}
```

Explanation:

If $i \leq j$, the value is stored.
Calculate the column-major index.
Return $A[\text{index}]$.
If $i > j$, return 0.

57. Full row-major helper example

```
#include <iostream>
using namespace std;

int getRowMajorIndex(int n, int i, int j) {
    if (i <= j) {
        return ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);
    }
    return -1;
}
```

```
int main() {
    int n = 5;

    cout << getRowMajorIndex(n, 4, 5) << endl; // 13
    cout << getRowMajorIndex(n, 5, 2) << endl; // -1

    return 0;
}
```

Expected output:

```
13
-1
```

58. Full column-major helper example

```
#include <iostream>
using namespace std;

int getColumnMajorIndex(int i, int j) {
    if (i <= j) {
        return (j * (j - 1) / 2) + (i - 1);
    }
    return -1;
}

int main() {
    cout << getColumnMajorIndex(4, 5) << endl; // 13
    cout << getColumnMajorIndex(5, 2) << endl; // -1

    return 0;
}
```

Expected output:

```
13
-1
```

59. Why get returns 0 but index helper returns -1

This is a common beginner confusion.

Helper index function

A helper index function returns an array index.

If the position is not stored, there is no valid array index.

So it returns:

-1

Get function

A get function returns the actual matrix value.

If the position is below the diagonal, the actual matrix value is:

0

So it returns:

0

Summary:

Function type	Not stored result	Why?
Index helper	-1	Means no valid array index
Get function	0	Actual matrix value is zero

60. Complexity notes

Storage complexity

An upper triangular matrix stores:

$N(N + 1) / 2$ values

This is less than N^2 , but in Big-O notation it is still:

$O(N^2)$

Why?

Because:

$N(N + 1) / 2 = (N^2 + N) / 2$

The largest growing term is N^2 .

So storage is:

$O(N^2)$, but optimized to about half of full storage

61. Time complexity of index calculation

Both formulas use direct arithmetic.

They do not use loops.

So index calculation takes:

$O(1)$

Row-major formula:

$\text{index} = ((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)$

Column-major formula:

$\text{index} = j(j - 1)/2 + (i - 1)$

Both are constant-time formulas.

62. Time complexity of set and get

A `set` operation does:

```
one condition check  
one index calculation if stored  
one array assignment if stored
```

So:

```
set() = O(1)
```

A `get` operation does:

```
one condition check  
one index calculation if stored  
one array lookup if stored
```

So:

```
get() = O(1)
```

63. Time complexity of display

Display must print the full visible matrix.

An `N x N` matrix has:

```
N² visible positions
```

So display uses nested loops:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // print value or zero  
    }  
}
```

```
}  
}
```

Therefore:

```
display() = O(N²)
```

Even though the storage is reduced, display still takes $O(N^2)$ time because the full matrix view has N^2 positions.

64. Complexity table

Operation	Time complexity	Space complexity	Reason
Storage	—	$O(N^2)$ optimized	Stores $N(N + 1)/2$ values instead of N^2
Index calculation	$O(1)$	—	Direct formula, no loop
set()	$O(1)$	—	One condition and one formula
get()	$O(1)$	—	One condition and one formula
display()	$O(N^2)$	—	Prints every visible matrix position

Part E: Important Formulas

65. Upper triangular condition formulas

Stored condition:

```
i <= j
```

Zero condition:

```
i > j
```

Stored values:

$$N(N + 1) / 2$$

Structural zeros:

$$N(N - 1) / 2$$

66. Upper triangular row-major formula

$$\text{index} = ((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)$$

Use only when:

$$i \leq j$$

Plain meaning:

Skip previous rows, then move right inside the current row.

67. Upper triangular column-major formula

$$\text{index} = j(j - 1)/2 + (i - 1)$$

Use only when:

$$i \leq j$$

Plain meaning:

Skip previous columns, then move down inside the current column.

68. Formula reference table

Storage type	Formula	Use when
Upper row-major	$((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)$	$i \leq j$
Upper column-major	$j(j - 1)/2 + (i - 1)$	$i \leq j$

Part F: Common Beginner Mistakes

69. Mistake 1: Using the lower triangular condition

Wrong for upper triangular:

```
if (i >= j)
```

Correct for upper triangular:

```
if (i <= j)
```

Why?

Because upper triangular values are on or above the main diagonal.

70. Mistake 2: Storing values below the diagonal

Wrong idea:

```
M[4,2] should be stored.
```

Correct idea:

```
M[4,2] is below the diagonal.  
4 > 2.  
So M[4,2] is zero.
```

It should not be stored.

71. Mistake 3: Forgetting to check $i \leq j$ before using the formula

Wrong:

```
int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);  
return A[index];
```

Correct:

```
if (i <= j) {  
    int index = ((i - 1) * n) - ((i - 2) * (i - 1) / 2) + (j - i);  
    return A[index];  
}  
return 0;
```

Why?

Because if the position is below the diagonal, it has no stored array index.

72. Mistake 4: Mixing row-major and column-major formulas

Row-major formula:

```
((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)
```

Column-major formula:

```
j(j - 1)/2 + (i - 1)
```

Do not use the row-major formula if your array was filled column by column.

Do not use the column-major formula if your array was filled row by row.

The formula must match the storage order.

73. Mistake 5: Thinking storage is $O(N)$, not $O(N^2)$

A diagonal matrix stores:

N values

So diagonal storage is:

$O(N)$

But an upper triangular matrix stores:

$N(N + 1) / 2$ values

So upper triangular storage is:

$O(N^2)$

It is optimized compared to full storage, but it is still quadratic growth.

Part G: Beginner Mental Model

74. Simple mental model for upper triangular matrix

Think of the full matrix as a visible grid.

Example:

```
1  2  3  4
0  5  6  7
0  0  8  9
0  0  0 10
```

But the computer does not store the zeros below the diagonal.

It stores only:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

If someone asks for:

```
M[2,4]
```

The program checks:

```
2 <= 4
```

True, so it calculates the index and reads the array.

If someone asks for:

```
M[4,2]
```

The program checks:

```
4 <= 2
```

False, so it returns zero immediately.

75. One-line memory trick

For upper triangular matrices:

```
The useful values are above the diagonal, so row number is less than or equal to  
column number.
```

That gives the condition:

```
i <= j
```

76. Final Lesson 9 summary

Lesson 9 teaches that an upper triangular matrix stores only values on and above the main diagonal.

The stored condition is:

$$i \leq j$$

The zero condition is:

$$i > j$$

For an $N \times N$ upper triangular matrix:

$$\begin{aligned}\text{stored values} &= N(N + 1) / 2 \\ \text{structural zeros} &= N(N - 1) / 2\end{aligned}$$

There are two storage methods.

Row-major upper triangular storage

Store row by row.

Formula:

$$\text{index} = ((i - 1)n) - ((i - 2)(i - 1)/2) + (j - i)$$

Mental model:

Skip previous rows, then move right.

Column-major upper triangular storage

Store column by column.

Formula:

$$\text{index} = j(j - 1)/2 + (i - 1)$$

Mental model:

Skip previous columns, then move down.

Both formulas are used only when:

$i \leq j$

If:

$i > j$

then the position is a structural zero, so the matrix value is:

0

77. Practice questions

Question 1

For an upper triangular matrix, is $M[3, 5]$ stored?

Check:

$3 \leq 5$

Answer:

Yes, it is stored.

Question 2

For an upper triangular matrix, is $M[5, 3]$ stored?

Check:

$5 \leq 3$

Answer:

No, it is a structural zero.

Question 3

Find the row-major index of $M[4,5]$ in a 5×5 upper triangular matrix.

Answer:

index = 13

Question 4

Find the column-major index of $M[4,5]$ in an upper triangular matrix.

Answer:

index = 13

Question 5

How many values are stored in a 6×6 upper triangular matrix?

Formula:

$$N(N + 1) / 2$$

Substitute:

$$\begin{aligned} & 6(6 + 1) / 2 \\ &= 6(7) / 2 \\ &= 42 / 2 \\ &= 21 \end{aligned}$$

Answer:

21 values

78. Lesson 9 exit checklist

You have understood Lesson 9 if you can explain these without help:

- What an upper triangular matrix is
- Why the stored condition is $i \leq j$
- Why positions with $i > j$ are structural zeros
- How many values are stored: $N(N + 1)/2$
- How many zeros exist: $N(N - 1)/2$
- What row-major storage means
- What column-major storage means
- How to use the row-major formula
- How to use the column-major formula
- Why `get()` returns 0 for positions below the diagonal
- Why helper functions may return -1 for positions not stored
- How upper triangular formulas relate to lower triangular formulas